RED Programming Language

Group 2
Student Name
Leader: Dana Alotaibi
Sara Algarni
Shayma Aljuaid
Shatha Alshaikh
Lujain Algrni
Rifan Barayan
Wareef Alzubaidi

Table of Contents

Table of Figures

1. Introduction

The Red programming language has been known as the world's first full-stack language. This pioneering language has been designed with a comprehensive set of functionalities that enable it to seamlessly handle all aspects of software development, from front-end to back-end. Its unique architecture and capabilities make it an exceptional tool for software engineers and developers who seek to streamline their development process and enhance the quality of their output. Its versatility and efficiency make it a valuable asset in academic and business settings alike, where innovation and productivity are paramount. As we embark on this research journey, we will delve deeper into the intricacies and nuances of the Red programming language, exploring its features, capabilities, and potential applications in various domains of software development. The text below provides an outline of the history of Red, including its origins and purpose as envisioned by its creator. Additionally, it describes the method by which the language was implemented to provide its functionality and enable its execution process. Furthermore, we will delve into the BNF and EBNF of the different components that are integral to the evaluation of programs developed by the compiler. These components include identifiers, variable declarations, expressions, selection statements, and loops. The aim is to provide a detailed understanding of their functioning. Our next agenda is to assess the effectiveness of Red in terms of its readability, writability, and reliability. We will scrutinize the specific attributes that contribute to each of these criteria and discuss their impact on the overall evaluation. Following up we will delve into the diverse types of variables offered by Red, followed by an exploration of the array types that are available in Red, and the various types of scopes that can be utilized. Lastly, we will present a Red program that is designed to calculate the average, minimum, and maximum grades.

2. Background

2.1 Historical Overview of RED

Red programming language is a flexible and easy-to-learn programming language. It was developed in 2011 by Peter Kóša and Ladislav Mecir. The idea of creating the Red programming language originated from another programming language called REBOL, which was developed by Karl Anton Scotten. REBOL has a strong capability for direct user interaction and a wide range of data types for expressing data and programming instructions. It also has the ability to execute commands quickly (Red Programming Language, 2020).

REBOL has numerous advantages, but it also has drawbacks and challenges. For example, REBOL was closed source (Red by Example, 2019) and its library capabilities were limited, meaning that users needed to create their own libraries and program many functions and tools from scratch. This increases the time and effort required for users to develop and program applications. There are also other challenges, such as integration and language documentation.

Peter Kóša and Ladislav Mecir were users of the REBOL programming language and had extensive experience with it. They found it to be a language that required expertise, intelligence, and effort, and it was not easy to learn and develop. Therefore, they came up with the idea of creating a new programming language called RED, based on the original REBOL language. One of the goals of the Red programming language was to overcome the limitations of the REBOL programming language and create a fully integrated programming language, improving certain aspects and functionalities and providing additional features to make it easier to learn and use in large projects.The developers of the Red programming language aimed to create a language that combines simplicity and expressive power. Kóša and Mecir analyzed various programming languages, studied their strengths, fundamentals, and weaknesses. They identified the features, concepts, and functionalities they wanted to integrate into their new language to make it comprehensive for most programming concepts, such as dynamic typing, functional programming, and object-oriented programming, in a simple syntax.To build a solid foundation for their language, they drew inspiration from the C language. The Red programming language is characterized by its dynamic nature, providing an interactive development environment that allows developers to write and execute code

immediately. This dynamic nature was one of the fundamental principles guiding the development process of Red, ensuring the language's flexibility and responsiveness to user needs. With the increasing popularity of Red, communities of developers interested in it have emerged worldwide. This diverse community plays a role in shaping the language, developing libraries, frameworks, and tools, which undoubtedly enhances the capabilities of the Red programming language, making it more flexible and powerful.

One of the key points of the Red programming language is its compatibility with various systems. Starting with the Linux operating system, efforts have been made to ensure that Red runs on other major platforms such as Windows and macOS.This cross-compatibility allows developers to write code once and deploy it on multiple systems, making Red a practical and efficient choice for a variety of projects.

REDs ambitious goal is to create the worlds first full-stack language that can be used for tasks ranging from system programming to high-level.

The Red programming language consists of two parts: the Red/System and Red.

The Red programming language can be used for high-level programming (GUIs and DSLs) as well as low-level programming (operating systems and device drivers). High-level programming is a language that is not understood by the computer, and these languages are designed to be easily understood by humans because they are easy to learn and apply. They use programming syntax and commands that are similar to the language used by humans in speech (Red Programming Language, 2020).

Red's some main features are:

- Functional, imperative, reactive and symbolic programming

Interactive programming is a fast and intuitive model closely related to the data table model that you can directly utilize in red color. More precisely, it is a method of linking one or more object fields to other fields or general terms by specifying relationships within a block of programming instructions, whether it is a single expression or a complex multi-step calculation. If the value of the source field changes, the target value is updated immediately. The wonderful thing is that you don't need to invoke a function for that. Here's a simple example in red (Red Programming Language, 2020):

```
red>> a: make reactor! [x: 1 y: 2 total: is [x + y]]
```

```
== make object! [
    x: 1
    y: 2
    total: 3
]
red>> a/x: 5
== 5
red>> a/total
== 7
red>> a/y: 10
== 10
red>> a/total
== 15
```

and Here is a comparative example with a reactive GUI vs the non-reactive version:

The reactive version:

```
to-int: function [value [percent!]][to integer! 255 * value]
view [
    below
    r: slider
    g: slider
    b: slider
    base react [
```

```
        face/color: as-color to-int r/data to-int g/data to-int b/data
    ]
]
```

The non-reactive version:

```
to-int: function [value [percent!]][to integer! 255 * value]
view [
    below
    slider on-change [box/color/1: to-int face/data]
    slider on-change [box/color/2: to-int face/data]
    slider on-change [box/color/3: to-int face/data]
    box: base black
]
```

- Powerful pattern-matching Macros system

A macro functions as a transformation that takes a reference to source code as input. Typically, the values produced by a macro are used to replace the macro's name and parameters at the point of the call in your source code. However, it is possible to customize this behavior and exert greater influence over the macro using pattern matching macros, as detailed below.

```
#macro as-KB: func [n][n * 1024]
print as-KB 64
```

When the macro is expanded by the preprocessor, the above source code will result in:

```
print 65536
```

This kind of simple macro is referred to as a named macro in Red's jargon. These are some of the main features of Red, and there are many more as well. For instance, it offers a cross-platform native GUI system with a UI layout DSL and a drawing DSL (Red Programming Language, 2020).

Moreover, Red boasts powerful performance. Significant optimizations and developments have been made to enhance its execution speed. Additionally, it is cross-platform, meaning it can run on various operating systems, providing greater flexibility in using the Red programming language across different platforms. Furthermore, there have been improvements in the core libraries, including memory management in the Red programming language, aimed at enhancing performance and reducing memory consumption.

A simple Hello World would look like in Red programming languge (Red Programming Language, 2020) :

```
>> print "Hello World!"

  Hello World!
```

As previously mentioned, the Red programming language consists of two parts: Red/System and Red. Regarding Red/System, it resembles the C language with support for memory pointers and a highly limited set of data types. Red/System provides a tool for building the Red library and low-level programming capabilities. Additionally, it has the ability to link code and produce executable files. Currently, Red/System uses 8-bit character encoding (ASCII). Red/System will transition to UTF-8 source code encoding when suitable Unicode support becomes available.

Here are some practical aspects of constructing Red language statements:

String delimiters: double quotes:

```
"this is a string"

{This is

a multiline

string. }
```

Block of code delimiters: square brackets:

```
if a > 0 [print "TRUE"]



either a > 0 [print "TRUE"][print "FALSE"]



while [a > 0][print "loop" a: a - 1]
```

Path separator: slash (denotes a hierarchical relation):

```
  s: declare struct! [i [integer!] b [byte!]]

s/i: 123

s/b: #"A"
```

In the past few months, numerous fantastic features have been added to Red/System.

To ensure optimal dryness of the lexer code, the development of the new Red lexer requires intra-function decomposition capabilities for the low-level components. Subroutines have been introduced as a solution, functioning similarly to GOSUB instructions in Basic. These subroutines are defined as distinct code blocks within the function body, enabling them to be called like regular functions, albeit without parameters. As a result, they offer faster and lighter performance compared to real function calls, requiring only stack space for storing the return address (Red Programming Language, 2020).

The declaration syntax is straightforward:

```
<name>: [<body>]



  <name> : subroutine's name (local variable).

  <body> : subroutine's code (regular R/S code).
```

Here is a first example of a fictive function processing I/O events:

```
process: func [buf [byte-ptr!] event [integer!] return: [integer!]

    /local log do-error [subroutine!]

  ][

    log: [print-line [">>" tab e "<<"]]

    do-error: [print-line ["** Error:" e] return 1]

    switch event [

      EVT_OPEN  [e: "OPEN"  log unless connect buf [do-error]]

      EVT_READ  [e: "READ"  log unless receive buf [do-error]]
      EVT_WRITE [e: "WRITE" log unless send buf    [do-error]]
      EVT_CLOSE [e: "CLOSE" log unless close buf   [do-error]]
      default   [e: "<unknown>" do-error]

    ]

    0

  ]
```

Furthermore, several new extensions have been added to the Red/System path in the Red programming language.

As an initial step towards enabling upcoming partial IO parallelism and parallel processing, a basic low-level OS thread wrapper API has been internally integrated into the Red runtime. Additionally, a collection of atomic intrinsics has been included to facilitate the implementation of lock-free and wait-free algorithms in multi-threaded execution contexts.

Another improvement implemented currently is the enhancement of character arrays. Notably, the hidden size in index slot /0 has been removed. The size of literal arrays can now be determined solely by size, and keywords are resolved at compile time for /0 indexed access, rather than at runtime.

One notable addition worth mentioning is the support for binary arrays. These arrays can be utilized for storing byte-oriented tables or embedding arbitrary binary data in source code. For Example:

```
table: #{0042FA0100CAFE00AA}

  probe size? table                  ;-- outputs 9

  probe table/2                      ;-- outputs "B"

  probe as integer! table/2          ;-- outputs 66
```

The ability to aggregate variables and function parameters is another recent feature that has emerged. It is now possible to aggregate the type definition for local variables and function parameters.

For example:

```
 foo: func [

    src dst    [byte-ptr!]

    mode delta [integer!]

    return:    [integer!]

    /local

       p q buf  [byte-ptr!]

       s1 s2 s3 [c-string!]

  ]
```

Here is a list of other changes and fixes: (Red Programming Language, 2020)

- Alias fields with cross-references in structures defined in the same context are now allowed.

  For example:

```
a!: alias struct! [next [b!] prev [b!]]
b!: alias struct! [a [a!] c [integer!]]
```

• Special floating point literal -0.0 is now supported.

• In addition to 1.#INF for positive infinity, +1.#INF is now supported as a valid literal.

• Context-aware acquisition word parsing.

• New #inline directive for inlining assembly binary code.

• Support for the % and // operators on floating point types has been deprecated because they rely on relative support from the FPU and the results are not reliable on all platforms. From now on, use the fmod function instead.

2.2 Implementation Method of RED

A computer is only capable of understanding machine language, which is made up of zeros and ones. Programs, on the other hand, are written in high-level languages that are simple for people to grasp. Implementation methods are responsible for translating high-level language codes into machine language so that programs can be executed. Compiler and Interpreter are the two categories of implementation methods.

A compiler is software that reads source code written by high-level language analyzes it and converts it into machine code. It works as a translator that translates the instructions of high-level language to machine-level language at once. Compilers come in different forms, these include Cross compilers, just-in-time(JIT) compilers, ahead-of-time(AOT) compilers, and bootstrap compilers… etc (GeeksforGeeks,2023). A cross-compiler is a type of compiler that allows programmers to write code on one platform and execute it for a different platform (G., What is Cross Compiler? , 2022). A just-in-time compiler is a compiler that works by converting bytecode into an instruction set that can be read by a machine's processor and it's responsible for performance optimization (GeeksforGeeks,2023). Bootstrap Compiler, a compiler written by the source programming language itself (Reynolds, J. H., 2003). Ahead-of-time compilation involves translating source code into machine code before execution, reducing the workload required at run time (G., What is AOT and JIT Compiler in Angular ?, 2020).

In contrast, an interpreter is a program that translates high-level language into an intermediate language and then into machine language. It works by translating one statement at a time. There are different types of interpreters such as bytecode Interpreters, threaded code Interpreters, and abstract syntax tree Interpreters … etc. (G., Difference between Compiler and Interpreter. , 2023).

Comparing a compiler to an interpreter, compiled code runs much faster than Interpreted code. in addition, the compiler gives debugging tools to detect errors easily, but it can catch only syntax errors and some semantic errors. On the other side, programs written in an interpreted language are easier to debug. The compiler also helps the application by improving its security. Allowing the management of memory automatically to reduce memory error risks is a great feature of the interpreter (G., Difference between Compiler and Interpreter. , 2023).

As previously indicated, the Red programming language draws inspiration from the REBOL programming language. Nonetheless, their implementation approaches are distinct from each other. While Rebol is an interpreted language, Red compiles statically deduced code and uses an interpreter for the rest. Implementation of Red was developed for many years for the first time Red compiler was written in Rebol and then it became a bootstrapping compile. Both compiled and interpreted code complement each other, offering a highly flexible language while ensuring maximum performance. The upcoming implementation of a JIT compiler aims to achieve an ideal internal architecture. Figure 1 shows the Red implementation method, which includes compilation, interpretation, and JIT compilation, and is also written in Red(Nenad Rakocevic,2013).

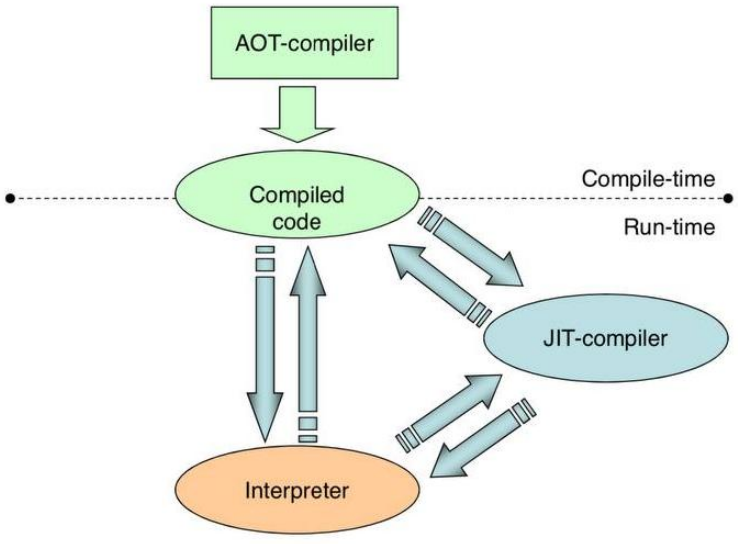

Figure 1:Red implementation method.

Compilation steps:

1-Lexical analysis: The initial stage of compilation processing involves lexical analysis, wherein the source code is meticulously scanned character by character and subsequently segmented into tokens based on the programming language's specified rules. The building blocks of the program's syntax, such as keywords, identifiers, punctuation, and constants are represented as tokens. The lexical analyzer ignores comments and white spaces as they do have not any use (Sebesta, 2009). A lexical analyzer is also referred to as a lexer or tokenizer (G., Lexical Analysis and Syntax Analysis., 2023). The process of Lexical Analysis in the Red programming language is carried out by the load function. Red has been using a lexer that was built entirely using the Parse dialect. This dialect was designed to prioritize ease of maintenance over performance. However, this approach has not been ideal, so Red considered rewriting the lexer entirely in Red/System for better performance. The new approach has a great feature, especially for performance. It is typically 50 to 200 times faster than the older one. In addition, the proposed solution aims to enhance the scanning process by accurately identifying the corresponding values along with their respective data types, thereby eliminating the need for loading them (Nenad Rakocevic,2020).

2-Syntax Analysis: In this step, the Syntax Analysis process compares the tokens generated in the previous step with the language rules to ensure they match them (G., Lexical Analysis and Syntax Analysis., 2023). After performing a comparison, any syntax errors in the lexical analysis tokens will be detected. Then the parser generates a Pars Tree that represents the logical structure of the program (Sebesta, 2009).

3-Semantic Analysis: The compiler utilizes both parse trees and symbol tables to identify the meaning of code and detect semantic errors. The symbol table is a crucial component in the compilation process, serving as a database that stores essential information. This information pertains to the type and attributes of every user-defined name present within the program (Sebesta, 2009). The following are the errors identified by the semantic analyzer: Type mismatch, Undeclared variables, and Reserved identifier misuse.

Semantic analysis has a variety of functions, such as:

- Type Checking: This refers to verifying that the data type being used corresponds to its definition and that the operator being applied has matching operands.
- Label Checking: Label references should be included in a program.
- Flow Control Check: It is important to ensure that control structures are used correctly.

(G., Semantic Analysis in Compiler Design., 2020)

4-Intermediate code generator and optimization: In this step, the compiler is responsible for generating intermediate code between the source code and the machine code. Optionally, the compiler may optimize the code to improve performance and reduce resource usage (Mogensen, T. G., 2017).

5-Code Generation: Finally, the compiler generates machine code that is translated to binary code for computer execution (Sebesta, 2009).

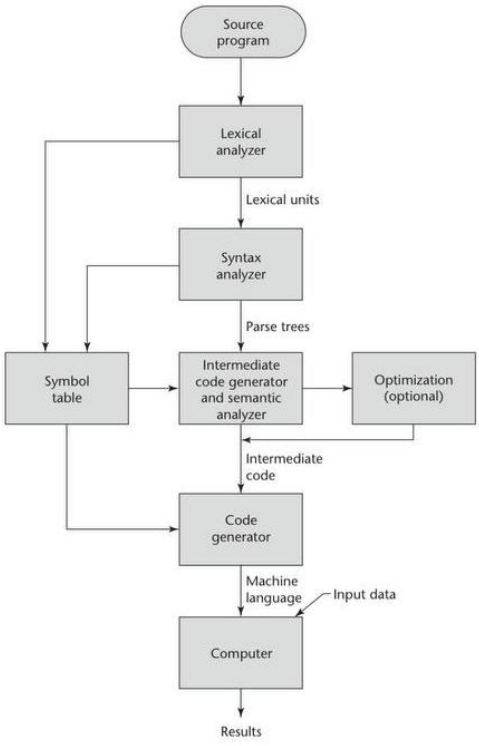

Figure 2:The Compilation Process
(Sebesta, 2009).

Interpretation steps:

The steps of Interpretation are much easier than the Compilation steps. As shown in Figure 4, started by passing the source program through the interpreter, and then the result appears directly (G., Difference between Compiler and Interpreter. , 2023).

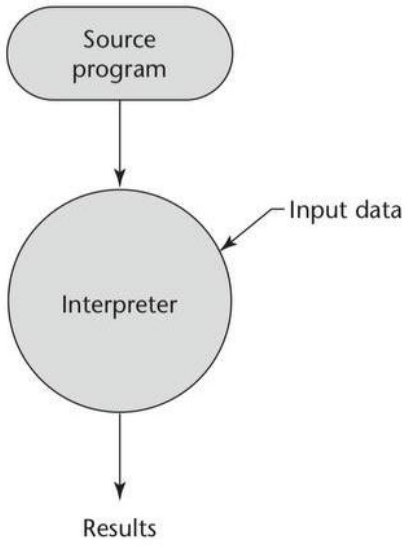

Figure 3:The Interpretation Process (Sebesta, 2009).

2.3 RED Application Domains

The Red programming language has many fields that have helped many people since its inception

Scientific field:

In

this field, people use Red programming in various specializations, including those working in academic settings and scientific research institutions. The Red programming language has multiple uses in the scientific field, such as data analysis and cloud modeling. It certainly accelerates discovery and innovation.

Data Analysis: Algorithms are developed to process difficult and complex data in order to show meaningful and useful results. Example in bioinformatics research, they analyze the data after they work to create custom programs to pre-process the raw sequence data and analyze gene expression patterns.

Cloud Modeling: It involves making mathematical models and studying real phenomena by simulating them. For example, in climate science research, they use red to predict the climate changes that will occur.

Healthcare field:

Infrared can be used in various programs, such as medical imaging, for example, to process and analyze images almost like X-rays, and it is also used in health care analysis to enhance the efficiency of health care delivery.

Game Development:

Although not as commonly used as other languages like C++ or Unity for game development, RED can still be employed for prototyping game mechanics, scripting game logic, and building simple games.

3. RED BNF/EBNF

3.1 What are BNF and EBNF?

BNF (Backus-Naur Form) and EBNF (Extended Backus-Naur Form) are notation systems that describe the syntax of the programming language. They are also considered as formal methods to specify the grammar rules of a programming language. (Biswas, 2023)

3.2 Identifiers

Identifiers are descriptive names used to identify program elements within code. They serve as the names for variables, functions, and other elements. Each programming language has its syntax for naming identifiers (Sebesta, 2009). In Red these are the rules for naming an identifier:

- **Length:** There are no restrictions on the length of words in Red language; they can be of any length.

- **Form:** An identifier must start with a letter (either uppercase or lowercase) or an underscore, followed by a sequence of letters, digits, and underscores.

- **Case sensitivity:** refers to a computer language's capability to recognize the difference between an upper- and lower-case form of a character. Red treats words as case insensitive, so 'color', 'Color', and 'COLOR' are considered the same identifiers. (Antonaccio, 2016)

- **Special words:** Red has reserved word that cannot be used as an identifier's name. For example, 'function' can only use to create a function. (Rakocevic, 2022)

➢ BNF for identifiers:

< identifier >: : = < letter > | "_" {< letter > | < digit > |"_}

< letter >: : = "a" | "b" | "c" | ⋯ | "z" | "A" | "B" | "C" | ⋯ | "Z"

< digit >: : = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

➢ EBNF for identifiers:

< identifier >: : = < letter > | "_" {< letter > | < digit > |"_"}

< letter >: : = "a" ⋯ "z" | "A" ⋯ "Z"

< digit >: : = "0" ⋯ "9"

3.3 Variable Declaration

Declaration of Variables is a fundamental concept in programming, where programmers define variables to store data within a program. (G., GeeksforGeeks, 2024) But in Red language do not need to be declared before being used, but require to be initialized anyway. (6_RED) So , When declaration the variables, we will assign a value for the variable without need to identifiers it with a specific datatype, here is the syntax of variable declaration in Red language :

VariableName: value

Let's represent this syntax in an example to illustrate the way it is used :
Name: Computer_Science22

As shown in the example above, declaring variables in Red is straightforward and unrestricted. Therefore, we simply need to write the variable name and its assigned value without specifying the data type, as it is automatically inferred.
Now, we will show the BNF of variable declaration in Red :

BNF of variable declaration:
< Declare > → < VariableName >
< VariableName > → name ":" value

We will apply our previous example on BNF:
Name: Computer_Science22
< Declare > → < VariableName >
→ Name : value
→ Name : Computer_Science22

3.4 Expressions

Programs are made up of expressions, which are combinations of values, variables, operators, and function calls. Language and operation play a big role in the complexity of expressions.

Programming language expressions can be divided into the following categories:

Values: Literal values, such as numbers or strings, can be included in expressions. An expression representing a specific value, such as 5, "hello", and true, is an example.

Variables: Expressions can contain variables, which are placeholders for values that may change during a program's execution. In this case, if x is a variable representing a number, then x + 5 is an expression involved in that variable.

Operators: A result is produced by operators, which operate on one or more operands. Examples of operators include logic operators (&&, ||, !), arithmetic operators (+, -, *, /), and comparison operators.(< ,> ,=! ,==)

Function Calls: The expression can contain a call to a function with arguments, which produces a result. Math.sqrt(25) calls the sqrt function from the Math class and evaluates to 5 as an example.

In the Red/System programs language, expressions serve as the building blocks.

They consist of: Sub-expressions in parentheses, variables, literal values, function calls, operator calls.

3.4.1 Formal grammar rules

BNF format is used to specify grammar rules, except when marking native language definitions with ...

```
<literal>       ::= ... any valid Red/System literal value ...
<variable>      ::= ... any valid Red/System variable name ...
<logic-call>    ::= ... function call that returns a value of logic! type ...
<func-call>     ::= ... function call that returns a value ...
<statement>     ::= ... statement or function without return value...

<logic-literal> ::= "true" | "false"

<math-op>       ::= "+" | "-" | "*" | "/" | "//" | "%"
<bitwise-op>    ::= "and" | "or" | "xor"
<comparison-op> ::= "=" | "<>" | "<" | "<=" | ">=" | ">"
<op>            ::= <math-op> | <bitwise-op>

<cond-expr>     ::= <value> <comparison-op> <expression>
<condition>     ::= <logic-literal> | <logic-call> | <cond-expr>

<all>           ::= "ALL" "[" <condition>+ "]"
<any>           ::= "ANY" "[" <condition>+ "]"
<short-circuit> ::= <all> | <any>

<paren>         ::= "(" <expression> ")"
<value>         ::= <variable> | <literal> | <paren> | "null"

<infix>         ::= <value> <op> <expression>
<prefix>        ::= <func-call> <expression>* | <statement> <expression>*
<call>          ::= <prefix>+ | <infix> | <cond-expr>

<expression>    ::= <call> | <value> | <short-circuit>
```

Figure 4: Formal grammar rules

In addition to being used standalone, expressions can also be used as arguments to other statements (RETURN, IF, or EITHER as statements).

Examples:

```
a: 123
foo a + 1
0 < foo a + 1
any [(0 < foo a + 1) a > 0]

if all [(0 < foo a + 1) a > 0][print "ok"]
```

Figure 5: Examples

3.4.2 Evaluation order rule

Expressions in Red language is evaluated from left to right.

Operational precedence is not present except for infix functions, which have precedence over prefixes. (Red/System Language Specification. (n.d.). https://static.red-lang.org/red-system-specs.html#section-4, n.d.)

Examples:

```
1 + 2 * 3                           ;-- (1 + 2) * 3 returns 9
1 + 2 * 3 = 9                       ;-- ((1 + 2) * 3) = 9 returns TRUE
9 = 1 + 2 * 3                       ;-- ((9 = 1) + 2) * 3 raises an error!

1 + (2 * 3)                         ;-- 1 + (2 * 3) returns 7

foo 1 + 2                           ;-- foo (1 + 2)
1 + foo 2 * 3                       ;-- 1 + (foo (2 * 3))
```

Figure 6: Examples how Expressions are evaluated

3.5 Selection (2-way if)

One of the most basic building blocks in any programming language is the if construct which lets you make decisions in your code. Almost every language out there has an if/else construct. Red is a bit different, it works like Rebol originally did (but was then changed when people asked for it). Fear not however, Red has a function called either which works exactly like Rebol's else refinement on the if function. (LearningRed_nd)

In programming, selections are implemented using IF-statements, and there are two types of it: one-way-if and two-way-if.

• **One-way if**: The statements inside the if-block are executed if the condition is satisfied (equal to true); if not, there is no other choice and the program will proceed to execute the remaining instructions.
The syntax of if statements in Red language is :

```
if (myvar = true) [
    ; do something here
]
```

Figure 7: syntax of if_ statements

An example of one-way if:

```
temperature: 25

; Using `if`
if temperature > 30 [
    print "It's hot outside!"
]
```

Figure 8: example of one-way if

Due to the temperature being set at 25, which is not higher than 30, the if statement's condition evaluates to false. It will thus not run the code block that prints "It's hot outside!" within the if expression.

Output : Since the print statement cannot be executed because of an unmet condition, there will be no results.

Flowchart for the if code:

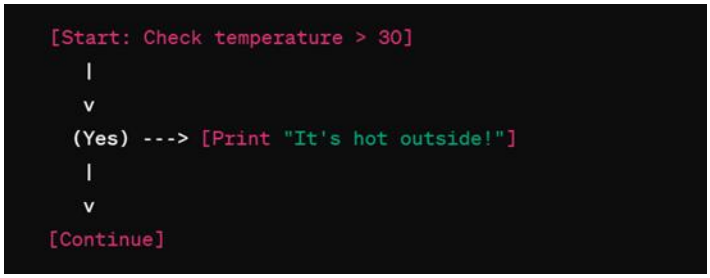

Figure 9:Flowchart of one-way if

• **Two-way-if :** It functions in the same manner as one-way-if, but In Two-way-if has a function called 'either' which works exactly like else function.

The '**either** ' statement in Red servers, has the same purpose as a two-way-if by allowing you to specify what happens in both the true and false cases of the condition.

The syntax of if-else statements (either) in Red language is :

```
either (myvar = true)[
    ; do something when true
] [
    ; do something when false
]
```

Figure 10:syntax of if_else statements(either)

An example of Tow-way-if:

```
temperature: 25

either temperature > 30 [
    print "It's hot outside!"
][
    print "It's not that hot outside."
]
```

Figure 11: example of Two-way if

Due to the temperature being set at 25, which does not fulfill the condition because 25 is not greater than 30, then we will ignore the first block (Which is achieved only in case the condition is right), execute the other block, and continue the remaining statements.

Output : It's not that hot outside.

Flowchart for the either code:

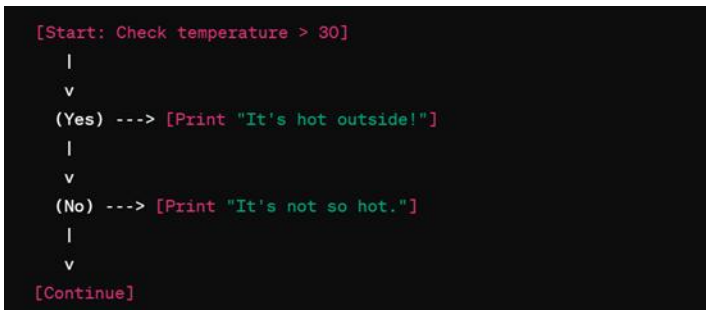

Figure 12:Flowchart of Two-way if

The BNF rules for the two-way if-else selection statement is displayed in the following table :

BNF of 2-way :
< Selection> := <if>|<either> <if> ::= "if" <condition> <block> <either> ::= "either" condition "["<block>"]" "[" <block> "]" <block> ::= "["statement* "]"

We will give an illustration of how to apply the BNF rules :

Temperature : 25

Either Temperature > 30 [

Print "It is hot outside"

][

Print "It is not hot outside"

]

< Selection → <either>

→ Either condition "["<block>"]" "[" <block> "]"

→ Either Temperature > 30 "[" <block> "]" "["<block>"]"

→ Either Temperature > 30 [<block>] "[" <block> "]"

→ Either Temperature > 30 [Print "It is hot outside"] "[" <block> "]"

→ Either Temperature > 30 [Print "It is hot outside"] [<block>]

→ Either Temperature > 30 [

→ Print "It is hot outside"

→] [

→ Print "It is not hot outside"

→]

3.6 Loob

Red like many programming languages, provides loop constructs for repeated execution of code blocks. The while loop is the primary looping construct in Red. It repeatedly executes a block of code as long as a specified condition remains true.

- **Syntax:**

While [< condition >] [<code>]

<condition> : a conditional expression

<code> : code to execute if the condition is met

Example:

```
c: 5
while [c > 5][
        Print “o”
        c: c-1
]
```

- **Loop Termination:**

You can achieve termination within the loop body by modifying the condition or using a break statement. Break allows to break out of the nearest enclosing loop at once and resume execution after the loop.

Red language uses free-form syntax. While there are no strict line breaks required, proper indentation is essential for readability and to define code blocks within loops. Consistent indentation helps visually identify the loop body.

4. Language Evaluation Criteria

Language evaluation criteria in programming languages are considered as the principles used to estimate the effectiveness of programming languages. They help evaluate the features, design, syntax, and semantics of programming languages aiming to determine their appropriateness for various applications and uses. When deciding on a programming language for a project, developers need to make sure they choose the best language by carefully assessing the criteria. (Mr.speed, 2023) There are several characteristics that affect the criteria, as shown in the figure.

	CRITERIA		
Characteristic	READABILITY	WRITABILITY	RELIABILITY
Simplicity	•	•	•
Orthogonality	•	•	•
Data types	•	•	•
Syntax design	•	•	•
Support for abstraction		•	•
Expressivity		•	•
Type checking			•
Exception handling			•
Restricted aliasing			•

Figure١٣ :Language Evaluation Criteria

4.1 Readability

Readability describes how easy and simple it is to read and comprehend the code written in a specific language. It affected by four of the characteristics which are simplicity, orthogonality, data types, and syntax design. (Sebesta, 2009)

4.1.1 Simplicity

Red language is known for its simplicity since it provides simple constructs that may be combined in many ways to create more complicated structures. Red also supports feature multiplicity by providing multiple ways to accomplish the same task for example you can assign a value to multiple variables in the same line which can reduce the readability of the language. (Rakocevic, 2022)

4.1.2 Orthogonality

Orthogonality means language structures must always work consistently and not impose unexpected restrictions for some of the actions (Sebesta, 2009). Here is some of the examples of orthogonality in Red:

Red language supports independent operation of several features, including functions, variables, and control structures. For example: definition of a function as in Figure١٤ :

```
>> ; Define a function anywhere in the code
>> multiply: func [a b] [ return a * b ]
== func [a b][return a * b]
>>
>> ; Call the function from any part of the program
>> mult: multiply 5 6
== 30
```

Figure١٤ : definition of a function

Red's syntax is consistent among many constructions. For instance, it is simple to comprehend and anticipate the syntax used to define variables, functions, and loops since it follows a consistent pattern.

```
>> ; Define a variable
>> x: 10
== 10
>>
>> ; Define a function
>> add: func [x y] [ return x + y ]
== func [x y][return x + y]
>>
>> ; Loop example
>> foreach item [1 2 3] [ print item ]
1
2
3
```

Figure١٥ : define variables, functions, and loops

In Red, some of the data type arguments are passed by value, such as logic! , integer!, and byte! , whereas c-string!, and struct! are passed by reference, which violates orthogonality. (Rakocevic, 2022)

4.1.3 Data Type

Red has a huge amount of data types which are used interchangeably, making coding more flexible and simpler. (amboris, 2024)

```
>> ; Define different data types
>> x: 10
== 10
>> y: "Hello"
== "Hello"
>> type? x
== integer!
>> type? y
== string!
>>
>> ; Change the values of one of them
>> x: 12.5
== 12.5
>> type? x
== float!
```

Figure١٦ : Example of changing the data type of a variable

In certain programming languages, the values one and zero are utilized to signify true and false. Whereas the Red language supports logic! data type that aids in differentiating between a literal one and one that signifies true. (amboris, 2024)

```
>> flag: true
== true
>> type? flag
== logic!
```

Figure١٧ : Logic! data type

4.1.4 Syntax Design

In the Red language, identifiers can be any length, which can impact readability but also allows for more descriptive variable names. The use of brackets to signify the end of a group can sometimes be confusing, especially in nested blocks. Programmers must pay attention to the structure of the program to easily identify where a block ends, ultimately affecting the readability of the code. (Blocks & Series, 2019)

4.2 Writability

Writability refers to how easily a programming language allows programmers to create programs for a chosen problem domain. It encompasses various characteristics that influence the ease and effectiveness of writing code. Red is designed with writability in mind, offering a simple and consistent syntax that is accessible to beginners and facilitates rapid development. The language emphasizes orthogonality, allowing various constructs to be combined and used interchangeably, enhancing expressiveness and code readability. With features like pattern matching, functional programming constructs, and a rich set of built-in functions, Red provides programmers with the tools to write concise and expressive solutions to problems across different domains.

4.3 Reliability

The reliability of RED programming language depends on what you're looking for in a language. Here's a breakdown of its strengths and weaknesses in terms of reliability:

Strengths:

- **Compiled language:** RED generates native code, which can be more reliable and performant compared to interpreted languages. This can lead to fewer runtime errors and crashes.
- **Zero-install, single file:** RED doesn't require complex installation or configuration. This reduces the possibility of errors due to dependency issues or incorrect setups.

Weaknesses:

- **Less mature:** Compared to mainstream languages, RED is a relatively new player. This means it might have fewer resources available for identifying and fixing bugs.
- **Smaller community:** A smaller developer base translates to less community support and troubleshooting resources. This can make it challenging to find solutions to problems you encounter.

Overall, RED can be a reliable option for small projects or personal use. However, if you need a highly stable and well-supported language for critical applications, a more established language like C or Python might be a better choice.

there are additional factors to consider:

- **Project requirements:** If your project demands maximum reliability and a large support network, RED might not be the ideal choice.
- **Your experience:** If you're comfortable working with less common languages and finding solutions independently, RED might be a good fit for you.

Ultimately, the best way to assess RED's reliability for your project is to try it out and see how it performs for your specific needs.

5. Types of Variables

5.1 Abstract Memory Cell

As a symbolic representation of the memory cell, variables in programming languages can be used to refer to specific locations in the memory of the computer. As a result of this abstraction, the intricate world of memory cells becomes more manageable, human-readable, and easier to understand.

The abstract memory cell is like a variable; it is made up of a collection of memory cells associated with a variable. Instead of dealing with raw addresses and numerical values, programmers can use named entities, making code much more intuitive and easier to comprehend.

5.2 Evolution from Machine Languages to Assembly

It was cumbersome, error-prone, and difficult to maintain working with absolute numeric memory addresses in the early days of programming. With the introduction of named variables, numeric addresses were replaced with meaningful identifiers during the transition to Assembly language.

Using variables in Assembly made it far easier for programmers to write and maintain programs. Instead of directly modifying memory cells through obscure addresses, they could use variable names, improving the overall clarity and maintainability of programs.

5.3 Sextuple of Variable Attributes

As abstractions of memory cells, variables have a set of attributes that define their behavior and use within a program. These attributes are as follows:

1. **Name:** The symbolic identifier assigned to the variable. In Red Language, The Variables are labels used to define a memory location. The labels (called identifiers from now on) consist of sequences of printable characters without any blanks (spaces, newlines, tabulations). In a system's console, printable characters are defined as any one-byte character in the 20h-7Eh range except for the following ones (used as delimiters or reserved for certain datatype literals):

```
[ ] { } " ( ) / \ @ # $ % ^ , : ; < >
```

Figure 18

The first character has a restriction, the following characters cannot be used in the first position, but are allowed in other positions:

```
0 1 2 3 4 5 6 7 8 9 '
```

Figure 19

There is another restriction to prevent the compiler from mistaking a hex integer for a variable name. Variable names starting with A-F letters must contain 2, 4 or 8 A-F and 0-9 characters. All identifiers (variables and function names) are case-insensitive.

Here is a list of reserved symbols and keyword that cannot be assigned to variables or functions:

```
%           &           *
+           -           -**
/           //          ///
<           <<          <=
<>          =           >
>>          >=          >>>
??          alias       all
and         any         as
assert      break       case
catch       comment     context
continue    declare     either
exit        false       func
function    if          loop
not         null        or
pop         push        return
size?       switch      throw
true        until       use
while       with        xor
```

Figure 20: Reserved symbols and keyword

2. **Address:** The address in memory where the variable is located.

3. **Value:** The current data stored in the variable. In most languages, the equality sign sets variables. In Red, the equality sign checks equality, and a colon sets the contents of the variable so you can assign a value to a variable using a colon character at the end of the variable identifier. This can be a real value (like an integer! or a pointer!) or a reference to the real value (like a struct! or a c-string!).

Examples:

```
>> course: "CPCS 301"
== "CPCS 301"
>> x: 6
== 6
>> y: 6.9
== 6.9
```

Figure ٢١: The ues of colon character in Red language

There is currently partial support for multiple assignments in Red/System through "assignment grouping". Essentially, it acts as a macro expansion where the pattern a: b: 123 would be expanded to a: 123 b: 123 and will duplicate whatever value is assigned to each variable.

Examples:

```
>> a: b: 123
== 123
>> print a
123
>> print b
123
```

Figure 22/Exapmles of Assignment grouping

4. **Type:** The data type of the variable (e.g., integer, float, string). In Red, each value is a specific datatype. The variables (words used to refer to values) are not strongly typed. Red is based on values, which have almost 50 different types. Many of these have unique, literal forms as well. (Red/System Language Specification. (n.d.). https://static.red-lang.org/red-system-specs.html#section-5, n.d.)

5.3.1 Primitive data

Primitive data types are the simplest data types supported directly by programming languages and do not consist of other types.

- **Integer!**

  Natural and negative natural numbers are supported by the integer! datatype. Integers have a 32-bit memory size, so the range of supported numbers includes:

  -2147483648 to 2147483647

Literal format:

```
decimal form          :  1234

decimal negative form : -1234

hexadecimal form      :  04D2h
```

Figure 23: Literal format

- **Byte!**

  An unsigned integer with a range between 0 and 255 is represented by this datatype.

Syntax:

```
#"<character>"
#"^<character>"
#"^(hexadecimal)"
#"^(name)"
```

Figure 24: Syntax of Byte! Data type

- **Float!**

  There is a 64-bit memory size for the float! datatype, representing a double precision floating point number according to IEEE-754.

Syntax:

```
<sign><digits>.<digits>
```

Figure 25: Syntax of Float! Data type

- **Float32!**

  This datatype represents an IEEE-754 single precision floating point number. Float32! memory size is 32-bits.

  One of the reasons for having a single precision floating point type is to simplify integrating it with popular libraries.

  For float32! datatype values, there is no literal form available. The loading method involves a float! literal prefixed with a float32! typecast.

Example:

```
pi32: as float32! 3.1415927
```

Figure 26: Example of float32! Data type

- **Logic!**

  As the name implies, logic! variables represent Boolean values: TRUE and FALSE. These variables are initialized by either entering a literal logic value or by executing a conditional expression.

Literal format:

```
true
false
```

Figure 27: Literal format of Logic! Data type

- **C-string!**

  C-strings are sequences of non-null bytes that end with a null byte. They hold the memory address of the first byte so you can think of them as implicit references to the byte! datatype.

  Double quotes are used to delimit literal C-strings, or a pair of curly braces:

Literal format:

```
foo: "I am a c-string"
bar: {I am
  a multiline
  c-string
}
```

Figure 28: Literal format of C-string! Data type

There is no need to add a null byte to literal c-strings. The null byte is added automatically during compilation.

5.3.2 User-Defined

The term "user-defined" is usually used in programming to refer to elements, entities, or components that are created or defined by the user (programmer)instead of predefined by the programming language. The user-defined elements allow programmers to customize solutions based on their specific needs.

- **Struct!**

  A struct variable is roughly equivalent to a C struct type. It is a record of one or several values, each with its own data type. A struct variable holds the memory address of a struct value.

```
declare struct! [
   <member> [<datatype>]
   ...
]
<member>   : a valid identifier
<datatype> : integer! | byte! | logic! | float! | float32! | c-
string! |
             pointer! [integer! | byte! | float! | float64! |
float32! | pointer!] |
             struct! [<members>] | struct! [<members>] value |
function! [<spec>]
```

Figure 29: Declare struct!

Struct values declared as local variables in functions are pre-reserved memory slots. DECLARE STRUCT! returns the memory address of the newly created struct! value. Whenever a new literal structs! is declared, all its members are initialized to zero.

- **Pointer!**

  Pointers are used to hold memory addresses of other values. They are defined by their address and datatype. Due to c-string! and struct! being implicit pointers, the only supported points are integer!, float!, float32!, and byte! (a logic! pointer is not required).

Literal format:

Syntax for creating a new pointer value is as follows:

```
declare pointer! [<datatype>]

<datatype>: integer! | byte! | float! | float32! | pointer!
```

```
foo: declare pointer! [integer!]
bar: declare pointer! [byte!]
baz: declare pointer! [float!]
qux: declare pointer! [pointer!]
```

Figure 30: Syntax for creating new pointer!

5. **Lifetime:** A variable's lifetime is how long it retains its value.
6. **Scope:** The part of the program in which the variable is accessible.

6. Array Types

In the red programming language, the concept of arrays is called Series and A block can be: [one block], [another block [a block within a block]]. There are differences in array from other programming languages. Among these differences is that the array starts from the number one and not from zero like other programming languages. This is because starting from zero is useless in the Red programming language. Also in the Red programming language we cannot create an array with a fixed size, unlike the Repoll language, and this can be considered a distinctive point in the Red language as determining the size may be postponed until the time of allocating data (Learning Red, 2018) .

Also in RED programming language, when adding and removing elements from an array it is very similar or identical to another programming language (Learning Red, 2018) .

An array can only be defined by creating an empty block such as:

```
my-array: copy[]
```

In order to access an element, we can use this formula

```
my-array/1: "Hello RED"

print my-array/1
```

To access its elements, you may use "/":

```
>> a/1
== [1 2]
>> a/1/1
== 1
>> a/3/2
== 6
```

Also In RED we can Using variable as keys for series to refer to the any element we need it ,you must use the :word syntax (Blocks & Series, 2019):

```
>> a: ["me" "you" "us" "them" "nobody"]
```

```
== ["me" "you" "us" "them" "nobody"]

>> b: 4

== 4

>> a/b        ;this does not work as expected!!!

== none

>> a/:b       ;this works!

== "them"
```

Here is an example of a 3 x 2 array:

```
>> a: [[1 2][3 4][5 6]]
== [[1 2] [3 4] [5 6]]
```

In RED programming language, we can perform some tasks for arrays, so here are some common typical tasks ((Red by Example, 2019))

sort a series :

```
red>> days: ["sun" "mon" "tue" "wed"]

   == ["sun" "mon" "tue" "wed"]

   red>> sort days

   == ["mon" "sun" "tue" "wed"]
```

find a value in series :

```
red>> days: ["sun" "mon" "tue" "wed"]

          == ["sun" "mon" "tue" "wed"]

          red>> find days "tue"

          "] ==tue" "wed["
```

7. Types of Scopes

In the world of programming, understanding variable scoping is crucial for crafting efficient and problem-free code. Variable scope sets the area or situation where a variable can be used, changed, or mentioned in a program. It creates limits that decide how long a variable last and where it can be seen. These limits are important because they impact how information is handled and shared in various sections of a code. There are two major types of variable scoping mechanisms: static scoping (lexical scoping) and dynamic scoping.

7.1 Static Scoping (Lexical Scoping)

Static scoping, also known as lexical scoping, is a widely used scoping mechanism in programming languages. It determines the scope of a variable based on its position in the source code, relying on the program's structure. In static scoping, a variable declared in an outer block is accessible to the inner blocks, but a variable declared in an inner block is not accessible to the outer blocks. This scoping mechanism provides clarity and predictability as programmers can determine the scope of a variable by examining the program's structure without executing the code. Static scoping helps prevent naming conflicts and allows for easier code maintenance.

7.2 Dynamic Scoping

Dynamic scoping is another scoping mechanism in programming languages, where the scope of a variable is determined by the current execution context. In dynamic scoping, the visibility and accessibility of a variable depend on the sequence of function calls at runtime. When a variable is referenced, the programming language searches for its value by traversing the call stack, starting from the current function and going up until the variable is found. Dynamic scoping offers flexibility, allowing variables to be accessed from any part of the code if they exist within the call stack. However, dynamic scoping can make the code harder to understand and maintain, as the scope of a variable cannot be determined solely by examining the program's structure.

7.3 Types of Scoping in Red

Red primarily follows static scoping principles. Static scoping determines variable visibility based on the block structure of the program. Variables declared in an outer block are accessible to inner blocks, while variables declared in an inner block are not accessible outside of that block. Static scoping provides predictability and clarity in terms of variable accessibility and helps prevent scope-related issues.

7.3.1 Global Context

Global context is defined as the global namespace where all global variables and functions are defined. This context is unique. As a simple rule, every variable not declared in a function is a global variable bound to global context.

Example:

```
foo: 123                        ;-- global variable

f: func [/local bar [integer!]][
   bar: 123                     ;-- locally scoped variable
]
```

7.3.2 Functions Context

Each defined function has its own local context. Variables declared in a function's definition block are locally scoped and can't be accessed outside of the function's body. On the other hand, global variable can be referenced and modified from functions. If a local variable has the same name as a global one, the local one will take precedence in function's body. The value of the homonym global variable won't be affected by local redefinitions in functions contexts.

Example:

```
foo: 1                          ;-- global variable
var: 2

f: func [
  return: [integer!]
  /local
     bar [integer!]
][
  bar: 3
  foo + var + bar                   ;-- will return 6
]

f: func [
  return: [integer!]
  /local
     bar [integer!]
     var [integer!]
][
  bar: 3
  var: 10                       ;-- 'var is a local variable here
  foo + var + bar                   ;-- will return 14
]
```

7.3.3 Namespaces

It is possible to define local namespaces to provide local contexts able to encapsulate source code.

- **Syntax**

```
<name>: context [<code>]

<name> : namespace identifier
<code> : body block of code
```

- Any code is allowed in the body block including function definitions. All variables and functions created will have a local name that can be accessed locally or from outside using context name prefix in a path notation:

```
<name>/<variable>                ;-- reading a variable from outside
<name>/<variable>:               ;-- setting a variable from outside
<name>/<func-name>                 ;-- invoking a function from outside
```

- The following elements are affected by namespace and become locally defined when declared in a context body block:

1- Variables
2- Functions
3- Imported functions
4- Enumerations
5- Aliases

Example:

```
b: 0
a: context [
  b: 123

  foo: func [/local b][
     b: 1
     print-line b
  ]

  print-line b                    ;-- will output 123
  foo                            ;-- will output 1
]

print-line b                      ;-- will output 0
print-line a/b                     ;-- will output 123
a/foo                             ;-- will output 1
a/b: a/b + 1
print-line a/b                     ;-- will output 124
```

Program

The program aims to read and process a set of grades to calculate their average. Additionally, the program is designed to identify the maximum and minimum grades within the set.

Code:

```
Red[]


; Function to read grades from the user

read-grades: func [][

    ;Ask user to enter number of grades and store the value in num-grades variable
    num-grades: to-integer ask "Enter Number of grades: "

    ;Loop to read grades
    repeat i num-grades [
        append grades to-integer ask rejoin ["Grade " i ": "]
    ]
]


; Function to calculate the average grade


average-grade: func [grades [block!]][
    sum: 0
    ;Loop to add grades
    foreach grade grades [
        sum: sum + grade
    ]

    ;calculate the average grade
    sum / length? grades
];end of function
```

Figure 31/The program

```
; Function to find the minimum grade
minimum-grade: func [grades [block!]][
    ; Initialize min-grade with the first grade
    min-grade: first grades

    ; Loop to check if there is a grade in the (grades) block less than min-grade
    foreach grade grades [
        if grade < min-grade [
            min-grade: grade
        ]
    ]

    ; return min-grade
    min-grade
];end of function


; Function to find the maximum grade
maximum-grade: func [grades [block!]][
    ; Initialize mAX-grade with the first grade
    max-grade: first grades

    ;Loop to check if there is a grade in the (grades) block greater than max-grade
    foreach grade grades [
        if grade > max-grade [
            max-grade: grade
        ]
    ]
    ; return min-grade
    max-grade
];end of function
```

Figure 32/Second part of the program

```


; Main program

; Create an empty block to store the grades
grades: []

;call read-grades function
read-grades
print []

;call average-grade function and send "grades" block to it then print the result
print  ["Average: " average-grade grades]
;call Maximum-grade function and send "grades" block to it then print the result
print  ["Maximum grade: " maximum-grade grades]
;call minimum-grade function and send "grades" block to it then print the result
print  ["Minimum grade: " minimum-grade grades]




;END OF THE PROGRAM
```

Figure 33/Third part of the program

Run:

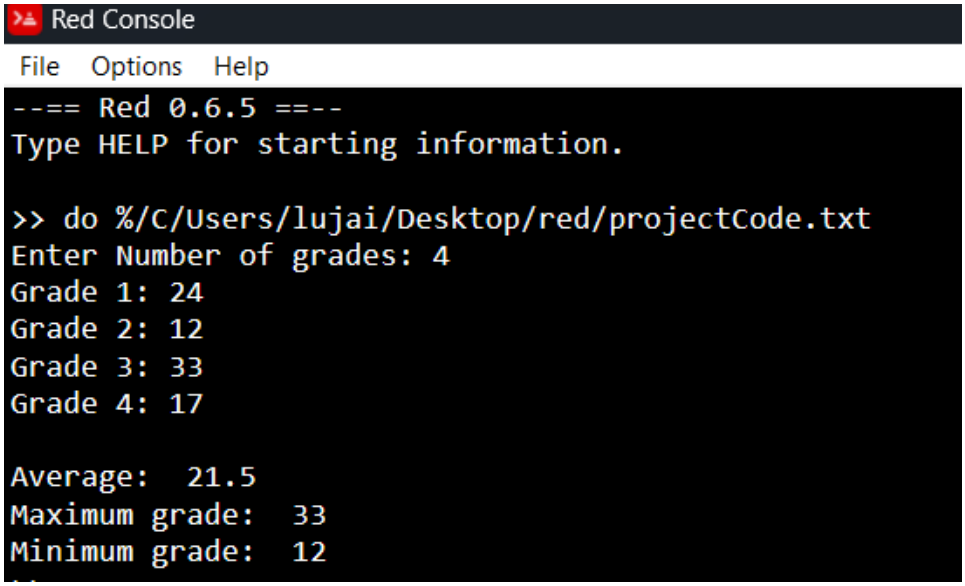

Figure 34/Sample run of the program

References

1. (https://www.geeksforgeeks.org/variables-programming/). (n.d.).
2. amboris. (2024, February 25). *red-docs-datatypes*. Retrieved from https://github.com/red/docs/blob/master/en/datatypes.adoc
3. Antonaccio, N. (2016, April 4). *Getting Started With Red*. Retrieved from https://chenyitian.gitbooks.io/getting-started-with-red/content/docs/fundamentals.html#33-variables
4. Biswas, A. (2023, July 17). *What are BNF and EBNF in Programming?* Retrieved from https://www.freecodecamp.org/news/what-are-bnf-and-ebnf/
5. *Blocks & Series*. (2019, January 27). Retrieved from Helpin' Red: https://helpin.red/BlocksSeries.html
6. G. (2020, April 22). *Semantic Analysis in Compiler Design.* Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/semantic-analysis-in-compiler-design/
7. G. (2020, November 5). *What is AOT and JIT Compiler in Angular ?* Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/what-is-aot-and-jit-compiler-in-angular/
8. G. (2022, November 28). *What is Cross Compiler?* . Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/what-is-cross-compiler/
9. G. (2023, January 24). *Lexical Analysis and Syntax Analysis.* Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/lexical-analysis-and-syntax-analysis/
10. G. (2023, June 12). *Difference between Compiler and Interpreter.* . Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/difference-between-compiler-and-interpreter/
11. G. (2023, May 11). *Introduction To Compilers.* Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/introduction-to-compilers/
12. G. (2024, March 19). *GeeksforGeeks*. Retrieved from Just In Time Compiler: https://www.geeksforgeeks.org/just-in-time-compiler/
13. https://crypticwyrm.neocities.org/learningred/#control-flow-ifelse, L. R. (n.d.).
14. https://static.red-lang.org/red-system-specs.html#section-3.1, R. L. (n.d.).
15. https://static.red-lang.org/red-system-specs.html#section-6.1. (n.d.).
16. https://www.geeksforgeeks.org/variables-programming/. (n.d.).
17. *Learning Red*. (2018, July 11). Retrieved from https://crypticwyrm.neocities.org/learningred/#viewing-images-with-help
18. Mogensen, T. G. (2017). *Introduction to Compiler Design.*
19. Mr.speed. (2023, April 8). *Language Evaluation Criteria*. Retrieved from https://www.geeksforgeeks.org/language-evaluation-criteria/

20. Rakocevic, N. (2022, september 8). Retrieved from Red/System Language Specification: https://static.red-lang.org/red-system-specs-light.html

21. *Red by Example*. (2019, Des 20). Retrieved from https://www.red-by-example.org/series.html

22. *Red Programming Language*. (2020, Aug 20). Retrieved from red: https://www.red-lang.org/search/label/red%2Fsystem

23. *Red/System Language Specification. (n.d.). https://static.red-lang.org/red-system-specs.html#section-4*. (n.d.).

24. *Red/System Language Specification. (n.d.). https://static.red-lang.org/red-system-specs.html#section-5*. (n.d.).

25. Reynolds, J. H. (2003, December 1). *Bootstrapping a self-compiling compiler from machine X to machine Y.* . Retrieved from https://doi.org/10.5555/948785.948811

26. Sebesta, R. W. (2009). *Concept Of Programming Language.* Pearson.

27. Umashankar, K. U. (2008). *The 8085 Microprocessor Architecture, Programming and Interfacing.* Dorling Kindersley (India).